UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

FACULTAD DE INGENIERÍA ELÉCTRICA,
ELECTRÓNICA, INFORMÁTICA Y MECÁNICA

ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA
Y DE SISTEMAS



IMPLEMENTACIÓN Y CONTROL DE UN BRAZO ROBÓTICO ARTICULADO DE 6 GDL

Asignatura: ROBÓTICA

Docente: Ing. Jose Mauro Pillco Quispe

Semestre: 2026-II

Integrantes:

- Salluca Chile, Sandro Alexander
- Choquenaira Quispe, Noe Franklin
- Ccasa Ccahuana, Flor Deliz
- Ramos Aguirre, Luis Alejandro

Cusco – Perú

2026

Índice

Resumen Ejecutivo **2**

1. Introducción y Objetivos **3**
1.1. Objetivos del Laboratorio . . . 3

2. Arquitectura General del Sistema y Hardware **3**
2.1. Distribución Cinemática de los 6 Grados de Libertad (6 GDL) . . . 3
2.2. Plataforma de Procesamiento: ESP32-CAM . . . 4

3. Análisis y Selección de Servomotores de Alto Par **4**

4. Ensamble Mecánico e Incidencia de Rigidez Articular **5**
4.1. Estructuración Física y Base Amortiguadora . . . 5
4.2. Diagnóstico de la Falla Mecánica: Pérdida de Rigidez en Hombro . . . 5
4.2.1. Plan de Acción y Corrección Inmediata . . . 6

5. Diseño Circuital y Plan de Conexionado Eléctrico **7**
5.1. Topología de Alimentación Desacoplada . . . 7
5.2. Asignación de Pines para el Bus I^2C . . . 8

6. Arquitectura del Firmware Embebido y Protocolos Web **8**
6.1. Estructuras de Datos y Cinemática Articular . . . 8
6.2. Mecanismos de Seguridad: Watchdog de Enlace . . . 8
6.3. Servidor Web Autónomo y API REST . . . 8

7. Código Fuente Completo del Firmware **9**

8. Pruebas Experimentales y Hoja de Ruta (Versión 2) **13**
8.1. Cronograma de Integración Física para la Versión 2 . . . 13

9. Conclusiones y Recomendaciones **13**
9.1. Conclusiones . . . 13
9.2. Recomendaciones . . . 14

Resumen Ejecutivo

El presente informe técnico documenta el diseño, reconfiguración mecatrónica, ensamble estructural y desarrollo de firmware para un **brazo robótico articulado de seis grados de libertad (6 GDL)**. Siguiendo las directrices del docente de la asignatura, el sistema fue rediseñado sustituyendo los servomotores iniciales por actuadores digitales de alto par torsor (**45 kg**, **40 kg** y **35 kg**), capaces de soportar los momentos estáticos generados por la masa acumulada del manipulador.

En este documento se expone con rigurosidad el estado de avance real del proyecto:

- **Ensamble Mecánico y Diagnóstico de Rigidez:** Se completó el montaje de los 6 grados de libertad sobre una base rígida de madera. Durante las primeras pruebas de elevación, se detectó una falla mecánica crítica: el brazo pierde sustentación y se desploma hacia adelante. La inspección visual determinó que el servocuerno de aluminio carece del tornillo central de apriete axial M3 sobre el eje estriado del servomotor negro de 45 kg (D2450ST), lo que ocasiona deslizamiento mecánico sin transmisión de par ni rigidez en la articulación del hombro.
- **Estado del Conexionado Eléctrico:** Se deja constancia de que el microcontrolador **ESP32-CAM** y el controlador PWM **PCA9685 aún no han sido conectados físicamente ni energizados**; se diseñó la topología circuital de masa común y aislamiento de rieles para su ejecución en la próxima sesión.
- **Firmware Embebido:** Se presenta y analiza el código fuente íntegro en C++ (`brazoR.ino`), el cual incorpora servidor HTTP embebido, comunicación I^2C a 50 Hz, algoritmos de movimiento suave e interrupción de seguridad (*watchdog* por inactividad).

Palabras clave: Brazo Robótico, 6 GDL, ESP32-CAM, PCA9685, Servomotores de Alto Par, Análisis de Falla Mecánica, UNSAAC.

1. Introducción y Objetivos

La robótica manipuladora serial constituye uno de los pilares formativos esenciales en la ingeniería mecatrónica e informática. Los brazos articulados de cadena cinemática abierta presentan retos considerables en el balance entre la rigidez estructural, la densidad de potencia de los actuadores y la precisión temporal en la modulación de señales de control.

1.1. Objetivos del Laboratorio

- **Objetivo General:** Implementar, ensamblar y programar un manipulador robótico de 6 grados de libertad teleoperado de manera inalámbrica mediante un microcontrolador ESP32-CAM y un controlador PWM periférico PCA9685.
- **Objetivos Específicos:**
  1. Reemplazar los actuadores convencionales por servomotores digitales de alto par torsor (45 kg, 40 kg y 35 kg) para satisfacer el balance estático del manipulador.
  2. Identificar y diagnosticar mediante inspección visual las holguras mecánicas y pérdidas de rigidez en las uniones hombro y codo.
  3. Diseñar un esquema circuital con aislamiento estricto entre el riel de alimentación lógica y el riel de potencia de los motores.
  4. Desarrollar y verificar el firmware en C++ para el microcontrolador ESP32-CAM con servidor web embebido y parada de emergencia.

2. Arquitectura General del Sistema y Hardware

El manipulador robótico antropomórfico está compuesto por tres subsistemas interconectados: subsistema mecánico, subsistema de control electrónico y subsistema de comunicaciones.

2.1. Distribución Cinemática de los 6 Grados de Libertad (6 GDL)

Cada eje de rotación cumple un rol geométrico determinado dentro del espacio de trabajo del efector final:

1. **Articulación 1 (Base - Azimuth):** Rotación sobre el plano horizontal para cobertura de 120°.
2. **Articulación 2 (Hombro - Pitch):** Elevación del brazo principal contra el vector de gravedad.
3. **Articulación 3 (Codo - Pitch):** Posicionamiento radial del antebrazo respecto a la base.
4. **Articulación 4 (Muñeca 1 - Pitch):** Inclinación vertical del cabezal de agarre.
5. **Articulación 5 (Muñeca 2 - Roll):** Giro axial de la pinza sobre su propio eje.
6. **Articulación 6 (Pinza - Gripper):** Prensión mecánica de objetos mediante mordazas paralelas.

2.2. Plataforma de Procesamiento: ESP32-CAM

El módulo ESP32-CAM integra un procesador de 32 bits Xtensa dual-core a 240 MHz con Wi-Fi 802.11 b/g/n y Bluetooth integrados. Previo al cableado de periféricos, se verificó el enlace USB-Serial con la base programadora ESP32-CAM-MB en el puerto `COM7` de Windows, confirmando la operatividad del silicio (Figura 1).



SÍ, LO RECONOCE. Sin dudas.

Acabo de comprobarlo en vivo, ahora mismo:

Prueba	Resultado
Windows lo ve	✅ Present: True, Status: OK
Puerto activo	✅ COM7 registrado
¿El chip contesta?	✅ SÍ – Connected to ESP32 on COM7
Chip	ESP32-D0WDQ6-V3
MAC	2c:bc:bb:84:a0:7c

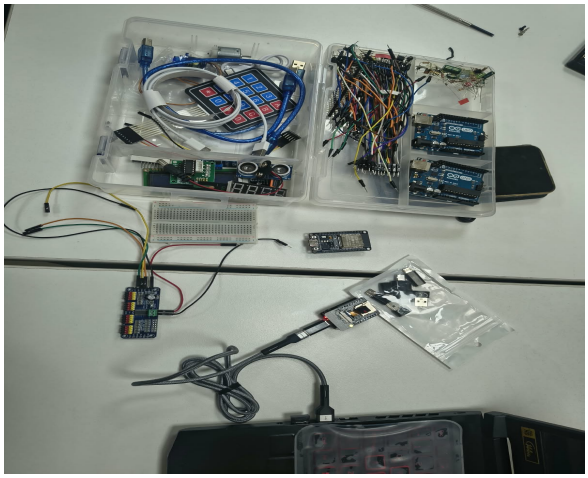
No es "aparece el puerto y ya". El ESP32 literalmente respondió y se identificó. Eso solo pasa si:

1. El cable transmite datos,
2. El CH340 está funcionando,
3. El driver está bien,
4. El ESP32 está vivo y alimentado.

Las cuatro cosas están bien. Ya puedes grabarle código desde Arduino IDE en el COM7.

Build · DeepSeek V4.1 Flash Free · 22.5s · 31.1 tok/s

(a) Diagnóstico UART: ESP32 en COM7 (MAC confirmada).



(b) Banco de pruebas: instrumentación y componentes.

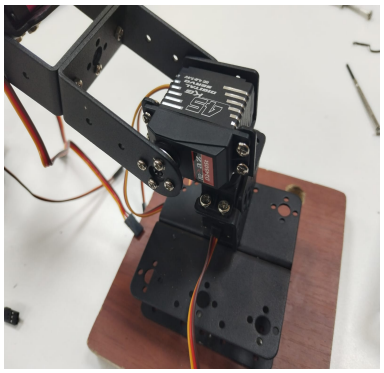
Figura 1: Verificación instrumental del microcontrolador y banco de trabajo en laboratorio.

3. Análisis y Selección de Servomotores de Alto Par

El cálculo del momento flector estático en la unión del hombro (J_2) demostró que los motores estándar de 10 kg · cm entraban en zona de bloqueo térmico. Siguiendo las instrucciones del docente, se reconfiguraron los actuadores según la solicitación mecánica de cada eslabón:

Tabla 1: Especificaciones Técnicas de los Servomotores Seleccionados

Ubicación	Modelo	Tensión / Señal	Par Torsor y Características
Eje 1: Base	DSSERVO 40 kg	DC 4.8V–6.8V	$\geq$ 40 kg · cm, piñonería de acero sobre rodamiento axial.
Eje 2: Hombro	D2450ST 45 kg	DC 4.8V–7.4V	$\geq$ 45 kg · cm, carcasa con aletas fresadas de disipación.
Eje 3: Codo	DSSERVO 40 kg	DC 4.8V–6.8V	$\geq$ 40 kg · cm, disipador fucsia, anclaje a eslabón tubular.
Eje 4: Muñeca 1	YSD035 35 kg Digital	DC 4.8V–6.0V	$\geq$ 35 kg·cm, placa anodizada azul, rango 500–2500 μs.
Eje 5: Muñeca 2	Digital 35 kg	DC 4.8V–6.0V	Giro axial del cabezal en soporte metálico.
Eje 6: Pinza	Metal Gear Mini	DC 4.8V–6.0V	Transmisión por doble engranaje para mordazas.



(a) Servo 45 kg D2450ST.



(b) Servo 40 kg DSSERVO.



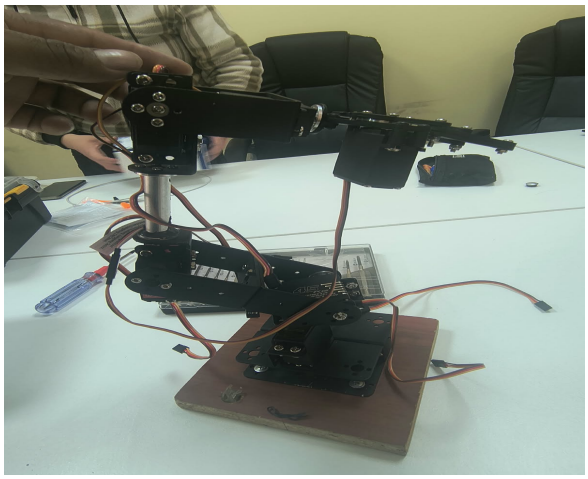
(c) Servo 35 kg YSD035.

Figura 2: Actuadores digitales de alta gama inspeccionados para la cadena cinemática.

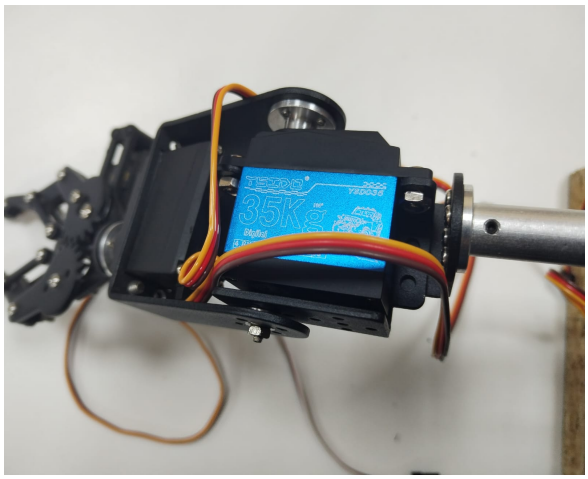
4. Ensamble Mecánico e Incidencia de Rigidez Articular

4.1. Estructuración Física y Base Amortiguadora

El ensamble del chasis se efectuó empleando perfiles de aleación de aluminio anodizado negro y tornillería métrica M3. Para garantizar la inercia basal y evitar desplazamientos parásitos por el centro de masa dinámico, la estructura completa se apernó a una plataforma de madera aglomerada de alta densidad (Figura 3).



(a) Estructura general de 6 GDL erguida.

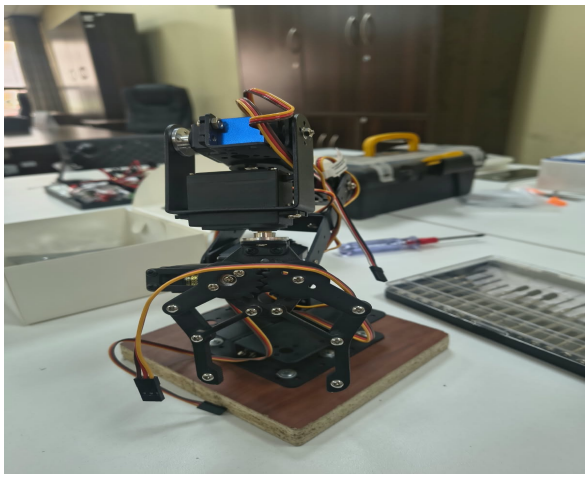


(b) Articulación de muñeca y eslabón cilíndrico.

Figura 3: Montaje de los eslabones mecánicos y articulaciones sobre la base fija.

4.2. Diagnóstico de la Falla Mecánica: Pérdida de Rigidez en Hombro

Durante la evaluación de sustentación mecánica en el laboratorio, se presentó un comportamiento anómalo grave: al situar el brazo en posición erguida, la articulación del hombro accionada por el servomotor negro de 45 kg (D2450ST) se mostró completamente suave y sin retención, careciendo de rigidez mecánica y causando el colapso y caída del manipulador hacia adelante bajo la sola acción de la gravedad (Figura 4).



(a) Caída frontal del brazo sobre la plataforma.

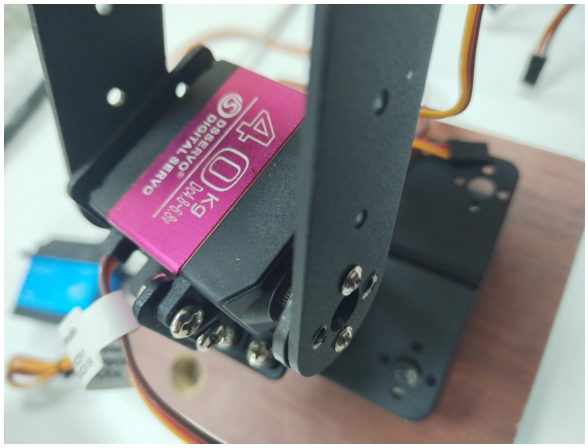


(b) Pérdida de ángulo y flexión libre del eje.

Figura 4: Manifestación física de la falla: colapso del brazo por ausencia de rigidez en el servomotor de 45 kg.

Incidencia Crítica de Laboratorio: Ausencia del Perno Axial Central en el Servomotor Negro de 45 kg (D2450ST)

La inspección visual microscópica (Figura 5a) reveló que el disco circular de aluminio (servocuerno) fijado al bracket en U del hombro **carece del tornillo métrico central que lo une axialmente con el eje estriado del servomotor negro de 45 kg (D2450ST)**. Al no existir este tornillo central, los dientes estriados del eje del servomotor patinan en vacío dentro del orificio del servocuerno. Por ende, aunque el motor posea un par nominal de 45 kg · cm, este no se transmite mecánicamente al eslabón del hombro, provocando que la unión se sienta sumamente suave y se produzca la caída inevitable del brazo.



(a) Macrofotografía: orificio central sin tornillo axial.



(b) Ajuste y alineación con destornillador en UNSAAC.

Figura 5: Causa raíz identificada y procedimiento de corrección mecánica en banco de trabajo.

4.2.1. Plan de Acción y Corrección Inmediata

Para subsanar definitivamente este fallo previo a la energización:

1. Instalar el tornillo axial M3 con arandela de seguridad en el servomotor negro de 45 kg del

hombro, así como en los servomotores de codo (40 kg) y muñeca (35 kg).

2. Aplicar fijador químico de roscas de resistencia media para mitigar desajustes por vibración.
3. Calibrar mecánicamente la postura de reposo a 90° antes de fijar los cuatro pernos perimetrales.

5. Diseño Circuital y Plan de Conexionado Eléctrico

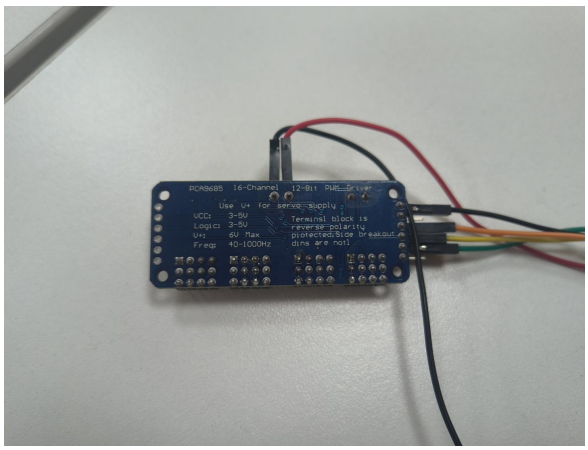
Estado Actual en Laboratorio (v2): Conexión Física Pendiente para la Siguiente Sesión

Se deja constancia explícita de que, a la fecha del presente informe (v2), **el ESP32-CAM y el módulo PCA9685 aún no han sido interconectados físicamente**. Los componentes se encuentran inspeccionados y los códigos de diagnóstico preparados para su ensamblado eléctrico en la próxima clase.

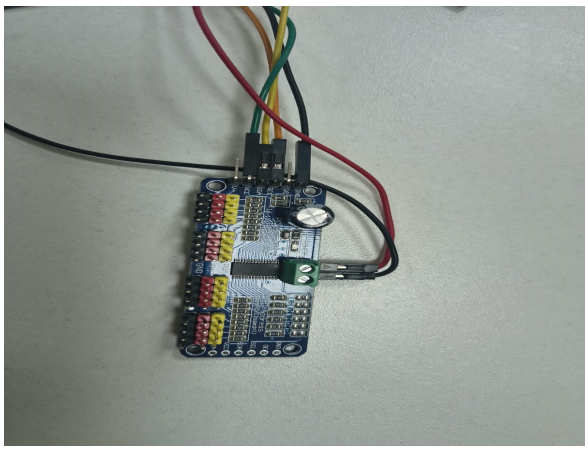
5.1. Topología de Alimentación Desacoplada

El accionamiento simultáneo de servomotores de 35 kg a 45 kg genera picos de corriente inductiva que superan los 4 A. Con el objetivo de proteger la etapa lógica del ESP32-CAM y prevenir reinicios involuntarios (*brownout*), se diseñó la siguiente arquitectura:

- **Alimentación Lógica (3.3V / 5.0V):** Entregada por el puerto USB de programación hacia el microcontrolador y el pin VCC lógico del PCA9685.
- **Alimentación de Potencia ($V_{ext} = 6.0$V):** Suministrada por una fuente de alimentación conmutada externa de 6 A hacia la bornera verde de tornillo del PCA9685. El condensador electrolítico integrado de 1000 μF suprime los transitorios rápidos.
- **Referencia Común (Star-GND):** Unión en un único punto del pin GND del microcontrolador con el borne GND de potencia del PCA9685 (Figura 6).



(a) Reverso del PCA9685: límites $V+ \leq 6$V.



(b) Líneas I2C y bornera de alta corriente.

Figura 6: Controlador periférico dedicado PCA9685 de 16 canales y 12 bits de resolución.

5.2. Asignación de Pines para el Bus I^2C

La interconexión serie entre el procesador y el driver PWM se estableció en terminales seguros que no interfieren con la cámara OV2640:

Tabla 2: Tabla Planificada de Conexiones entre ESP32-CAM y PCA9685

Pin ESP32-CAM	Pin PCA9685	Nivel Lógico	Función y Rol Técnico
GPIO 13	SDA	3.3V CMOS	Línea bidireccional de datos del bus I^2C
GPIO 14	SCL	3.3V CMOS	Línea de reloj síncrono del bus I^2C
VCC (5V / 3.3V)	VCC	3.3V / 5.0V	Alimentación del núcleo digital del integrado
GND	GND	0V	Masa común digital y de potencia
Fuente Ext. (+)	Bornera V+	6.0V DC	Riel de potencia exclusivo para servomotores
Fuente Ext. (−)	Bornera GND	0V	Retorno de corriente de los 6 actuadores

6. Arquitectura del Firmware Embebido y Protocolos Web

El firmware implementado en C++ (`brazoR.ino`) gestiona concurrentemente la generación de pulsos PWM sobre el PCA9685 y el servicio web HTTP interactivo.

6.1. Estructuras de Datos y Cinemática Articular

Cada articulación del robot se modela mediante la estructura `Joint`, garantizando límites angulares seguros por software para proteger los eslabones metálicos:

- **Canales PWM:** Asignados correlativamente del 0 al 5 en el PCA9685.
- **Rango Seguro:** Acotado estrictamente en $[30°, 150°]$ con posición de reposo (*HOME*) en 90°.
- **Ancho de Pulsos:** Rango calibrado de 1000 μs a 2000 μs (ampliable a 500–2500 μs según motor).

6.2. Mecanismos de Seguridad: Watchdog de Enlace

Para prevenir accidentes en caso de que el operador pierda la cobertura Wi-Fi:

- **Temporizador de Heartbeat (`LINK_TIMEOUT_MS = 10000`):** Si transcurren más de 10 segundos sin recibir peticiones HTTP, se dispara automáticamente la rutina `stopAll()`.
- **Corte de Pulsos (`stopAll()`):** Escribe el bit `FULL_OFF` mediante `pwm.setPWM(ch, 0, 4096)`, suspendiendo la modulación para evitar sobrecalentamiento en los motores.

6.3. Servidor Web Autónomo y API REST

El ESP32-CAM inicializa un Punto de Acceso (*SoftAP*) con SSID `Brazo-ESP32CAM` en la dirección IP `192.168.4.1`. La página web se aloja comprimida en memoria de programa (`PROGMEM`) en `pagina.h`, consumiendo los siguientes endpoints REST:

- `GET /api/status`: Telemetría completa de ángulos y estados en formato JSON.
- `POST /api/servo`: Actualiza el ángulo objetivo de una articulación específica.
- `POST /api/enable`: Activa o desactiva individualmente el pulso de un servo.
- `POST /api/home`: Envía los servos activos a su postura central de 90°.
- `POST /api/stop`: Parada de emergencia instantánea en todos los canales.

7. Código Fuente Completo del Firmware

A continuación se transcribe el código fuente completo del firmware desarrollado para el microcontrolador ESP32-CAM (`codigo/firmware_brazo_esp32/brazoR.ino`):

```
#include <Arduino.h>
#include <Wire.h>
#include <WiFi.h>
#include <WebServer.h>
#include <Adafruit_PWMServoDriver.h>

#undef HTTP_ANY
#include "camara.h"
#include "pagina.h"

// Credenciales del Punto de Acceso (SoftAP)
constexpr char AP_NAME[] = "Brazo-ESP32CAM";
constexpr char AP_PASSWORD[] = "CambiaEstaClave123";
constexpr uint8_t SDA_PIN = 13;
constexpr uint8_t SCL_PIN = 14;
constexpr uint8_t PCA_ADDRESS = 0x40;
constexpr uint32_t LINK_TIMEOUT_MS = 10000;
constexpr uint32_t MOTION_INTERVAL_MS = 20;

struct Joint {
  const char *name;
  uint8_t channel;
  uint8_t minAngle;
  uint8_t maxAngle;
  uint8_t home;
  uint16_t minPulseUs;
  uint16_t maxPulseUs;
  uint8_t current;
  uint8_t target;
  bool enabled;
};

// Limites seguros de operacion: calibrar previo a montaje definitivo
Joint joints[] = {
    {"Base",       0, 30, 150, 90, 1000, 2000, 90, 90, false},
    {"Hombro",     1, 30, 150, 90, 1000, 2000, 90, 90, false},
    {"Codo",       2, 30, 150, 90, 1000, 2000, 90, 90, false},
    {"Muneca 1",   3, 30, 150, 90, 1000, 2000, 90, 90, false},
    {"Muneca 2",   4, 30, 150, 90, 1000, 2000, 90, 90, false},
    {"Pinza",      5, 30, 150, 90, 1000, 2000, 90, 90, false},
};
constexpr size_t JOINT_COUNT = sizeof(joints) / sizeof(joints[0]);

Adafruit_PWMServoDriver pwm(PCA_ADDRESS, Wire);
WebServer server(80);
bool driverReady = false;
uint32_t lastHeartbeat = 0;
uint32_t lastMotion = 0;

bool parseNumber(const String &value, int &out) {
  if (value.isEmpty() || value.length() > 3) return false;
  int number = 0;
  for (size_t i = 0; i < value.length(); ++i) {
    if (value[i] < '0' || value[i] > '9') return false;
    number = number * 10 + (value[i] - '0');
  }
  out = number;
  return true;
}

void json(int code, const String &body) {
  server.sendHeader("Cache-Control", "no-store");
  server.sendHeader("X-Content-Type-Options", "nosniff");
  server.send(code, "application/json; charset=utf-8", body);
}

void stopAll() {
  if (!driverReady) return;
```

```
69    for (Joint &joint : joints) {
70      pwm.setPWM(joint.channel, 0, 4096); // FULL_OFF
71      joint.enabled = false;
72      joint.target = joint.current;
73    }
74  }
75
76  void writeAngle(Joint &joint) {
77    const uint16_t pulse = map(joint.current, 0, 180,
78                               joint.minPulseUs, joint.maxPulseUs);
79    pwm.writeMicroseconds(joint.channel, pulse);
80  }
81
82  bool selectedJoint(Joint *&joint) {
83    int index;
84    if (!server.hasArg("i") || !parseNumber(server.arg("i"), index) ||
85        index >= static_cast<int>(JOINT_COUNT)) {
86      json(400, "{\"error\":\"Indice de servo invalido\"}");
87      return false;
88    }
89    joint = &joints[index];
90    return true;
91  }
92
93  bool requireDriver() {
94    if (!driverReady) {
95      json(503, "{\"error\":\"PCA9685 no detectado: revisar conexion I2C\"}");
96      return false;
97    }
98    return true;
99  }
100
101 void sendStatus() {
102   String body;
103   body.reserve(780);
104   body = "{\"ready\":";
105   body += driverReady ? "true" : "false";
106   body += ",\"camera\":";
107   body += cameraReady ? "true" : "false";
108   body += ",\"psram\":";
109   body += psramReady ? "true" : "false";
110   body += ",\"servos\":[";
111   for (size_t i = 0; i < JOINT_COUNT; ++i) {
112     const Joint &joint = joints[i];
113     if (i) body += ',';
114     body += "{\"name\":\"";
115     body += joint.name;
116     body += "\",\"angle\":";
117     body += String(joint.current);
118     body += ",\"target\":";
119     body += String(joint.target);
120     body += ",\"min\":";
121     body += String(joint.minAngle);
122     body += ",\"max\":";
123     body += String(joint.maxAngle);
124     body += ",\"enabled\":";
125     body += joint.enabled ? "true" : "false";
126     body += '}';
127   }
128   body += "]}";
129   json(200, body);
130 }
131
132 void enableJoint() {
133   if (!requireDriver()) return;
134   Joint *joint;
135   if (!selectedJoint(joint)) return;
136   int on;
137   if (!server.hasArg("on") || !parseNumber(server.arg("on"), on) || on > 1) {
138     json(400, "{\"error\":\"on debe ser 0 o 1\"}");
139     return;
140   }
141   if (on) {
```

```
    if (!joint->enabled) {
      joint->current = joint->home;
      joint->target = joint->home;
      writeAngle(*joint);
      joint->enabled = true;
    }
    lastHeartbeat = millis();
  } else {
    pwm.setPWM(joint->channel, 0, 4096);
    joint->enabled = false;
    joint->target = joint->current;
  }
  sendStatus();
}

void setAngle() {
  if (!requireDriver()) return;
  Joint *joint;
  if (!selectedJoint(joint)) return;
  int angle;
  if (!server.hasArg("angle") || !parseNumber(server.arg("angle"), angle) ||
      angle < joint->minAngle || angle > joint->maxAngle) {
    json(400, "{\"error\":\"Angulo fuera de los limites configurados\"}");
    return;
  }
  if (!joint->enabled) {
    json(409, "{\"error\":\"Activar primero este servo\"}");
    return;
  }
  joint->target = angle;
  lastHeartbeat = millis();
  sendStatus();
}

void home() {
  if (!requireDriver()) return;
  for (Joint &joint : joints) {
    if (joint.enabled) joint.target = joint.home;
  }
  lastHeartbeat = millis();
  sendStatus();
}

void moveServos() {
  const uint32_t now = millis();
  bool anyEnabled = false;
  for (const Joint &joint : joints) anyEnabled |= joint.enabled;
  if (anyEnabled && now - lastHeartbeat >= LINK_TIMEOUT_MS) {
    stopAll();
    Serial.println("Sin latido del navegador: PWM desactivado");
  }
  if (now - lastMotion < MOTION_INTERVAL_MS) return;
  lastMotion = now;
  for (Joint &joint : joints) {
    if (!joint.enabled || joint.current == joint.target) continue;
    joint.current += (joint.current < joint.target) ? 1 : -1;
    writeAngle(joint);
  }
}

void setup() {
  Serial.begin(115200);
  startCamera();
  Wire.begin(SDA_PIN, SCL_PIN);
  Wire.setClock(100000);
  driverReady = pwm.begin();
  if (driverReady) {
    pwm.setPWMFreq(50);
    stopAll();
  } else {
    Serial.println("PCA9685 no detectado en 0x40. Revisar SDA/SCL, GND y VCC.");
  }

```

```
  WiFi.mode(WIFI_AP);
  WiFi.softAPConfig(IPAddress(192, 168, 4, 1), IPAddress(192, 168, 4, 1),
                    IPAddress(255, 255, 255, 0));
  if (!WiFi.softAP(AP_NAME, AP_PASSWORD)) {
    Serial.println("ERROR: no se pudo crear la red Wi-Fi");
    return;
  }
  server.on("/", HTTP_GET, []() {
    server.sendHeader("Cache-Control", "no-store");
    server.send_P(200, "text/html; charset=utf-8", INDEX_HTML);
  });
  server.on("/api/status", HTTP_GET, sendStatus);
  server.on("/api/enable", HTTP_POST, enableJoint);
  server.on("/api/servo", HTTP_POST, setAngle);
  server.on("/api/home", HTTP_POST, home);
  server.on("/api/stop", HTTP_POST, []() { stopAll(); sendStatus(); });
  server.on("/api/heartbeat", HTTP_POST, []() {
    lastHeartbeat = millis();
    sendStatus();
  });
  server.onNotFound([]() { json(404, "{\"error\":\"Ruta no encontrada\"}"); });
  server.begin();
  startCameraStream();
  Serial.print("Conectar a ");
  Serial.print(AP_NAME);
  Serial.print("; abrir http://");
  Serial.println(WiFi.softAPIP());
}

void loop() {
  server.handleClient();
  moveServos();
}
```

Listing 1: Firmware completo del brazo robótico (brazoR.ino)

8. Pruebas Experimentales y Hoja de Ruta (Versión 2)

Durante la jornada de trabajo desarrollada en el laboratorio de robótica (Figura 7), los estudiantes completaron el análisis físico del prototipo y el dimensionamiento de las etapas de potencia.



Figura 7: Equipo de estudiantes durante la sesión de ensamblado y diagnóstico en laboratorio UNSAAC.

8.1. Cronograma de Integración Física para la Versión 2

En la siguiente sesión de laboratorio se ejecutará la secuencia de puesta en marcha definitiva:

1. **Corrección Mecánica de Articulaciones:** Colocación de los tornillos axiales M3 en los servocuernos de aluminio (priorizando el servomotor negro de 45 kg del hombro) para asegurar la transmisión rígida de par y eliminar el juego articular.
2. **Verificación del Bus I^2C:** Carga del programa de diagnóstico `codigo/prueba_i2c_pca9685/test_i2c_s` para confirmar el reconocimiento de la dirección `0x40` en los terminales GPIO 13 y 14.
3. **Conexión de la Fuente de Potencia:** Interconexión de la fuente conmutada de $6.0\text{V} - 6\text{A}$ a la bornera del PCA9685 con verificación previa de polaridad y aislamiento de masa común con el ESP32-CAM.
4. **Pruebas de Movimiento y Calibración Angular:** Ensayo de trayectorias articulares de forma individual (base, hombro, codo, muñeca y gripper) validando el consumo de corriente y los ángulos de fin de carrera.
5. **Integración del Streaming de Video:** Fusión del servidor MJPEG en puerto 81 dentro del dashboard web para visualización en tiempo real.

9. Conclusiones y Recomendaciones

9.1. Conclusiones

1. Se completó con éxito el diseño estructural y el ensamble mecánico preliminar de un manipulador serial de 6 grados de libertad, incorporando los servomotores de alto par torsor (**45 kg**, **40 kg** y **35 kg**) solicitados por el docente.

2. La inspección visual del prototipo permitió diagnosticar con precisión la causa de la pérdida de rigidez en la articulación de elevación: la omisión del perno axial central en el servocuerno circular del servomotor negro de 45 kg (D2450ST), el cual desacoplaba mecánicamente el eje dentado del motor del bracket en U.

3. Se verificó la operatividad del procesador **ESP32-CAM** en el puerto serial `COM7`, dejando la plataforma electrónica y el firmware preparados para la conexión física con el driver **PCA9685** en la siguiente sesión de laboratorio.

4. El firmware desarrollado en C++ incorpora mecanismos de seguridad avanzados (temporizador de latido y corte automático por pérdida de señal Wi-Fi) que mitigan riesgos de bloqueo o daño estructural.

9.2. Recomendaciones

1. **Seguridad Mecánica:** Asegurar todos los servocuernos metálicos con pernos axiales apretados a torque adecuado y utilizar arandelas de presión antes de energizar la etapa de potencia.

2. **Aislamiento de Potencia:** Conectar primero la masa común (GND) antes de suministrar tensión a la bornera $V+$ del PCA9685 para evitar corrientes de retorno a través de las líneas lógicas I2C.

3. **Prueba Escalonada:** Probar los servomotores uno a uno desde la interfaz web, manteniendo presionado el botón de parada de emergencia ante cualquier indicio de bloqueo cinemático.

Referencias

[1] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed., Pearson Prentice Hall, 2005.

[2] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*, John Wiley & Sons, 2006.

[3] Espressif Systems, *ESP32-CAM Development Board Documentation and Camera Driver*, 2024.

[4] NXP Semiconductors, *PCA9685: 16-channel, 12-bit PWM Fm+ I2C-bus LED driver Datasheet*, Rev. 4, 2015.

[5] Adafruit Industries, *Adafruit PWM Servo Driver Library Documentation*, 2023.